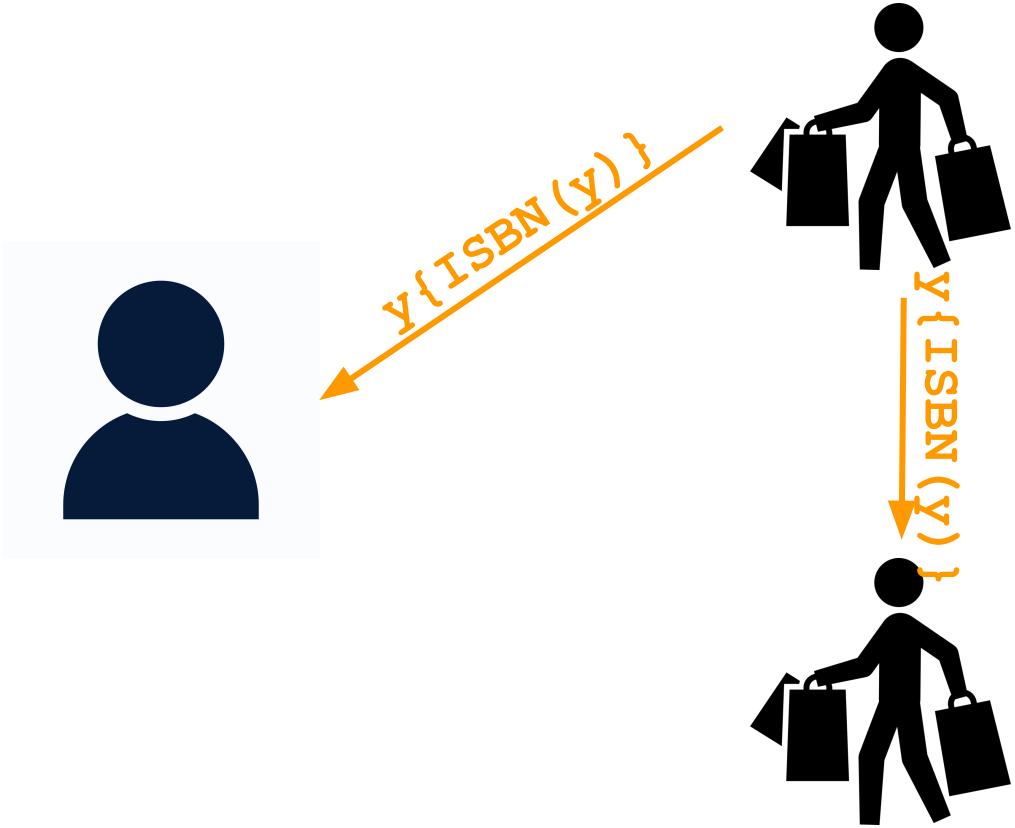
Decision Problems for Global Protocol Specifications

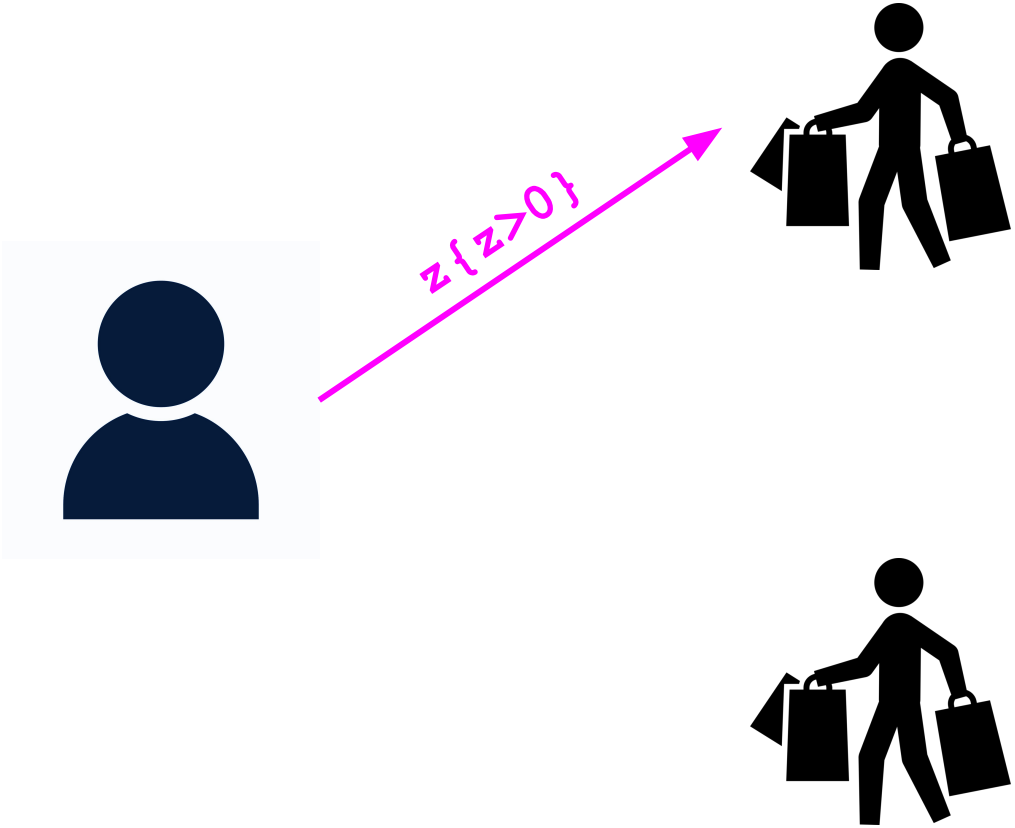
Elaine Li

Monday, 4th August

Example: two-bidder protocol



Example: two-bidder protocol



Example: two-bidder protocol

<z1,z2>
{z1,z2>0}

b1{b1>z1}

Example: two-bidder protocol

<z1,z2>

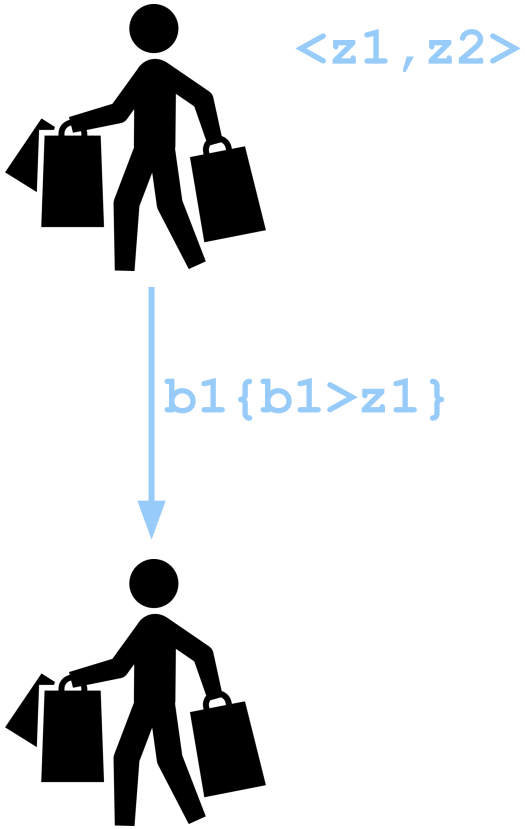
b2{b2>z2}

Example: two-bidder protocol

<z1:=b1,z2:=b2>

cont{b1+b2<z}

Example: two-bidder protocol

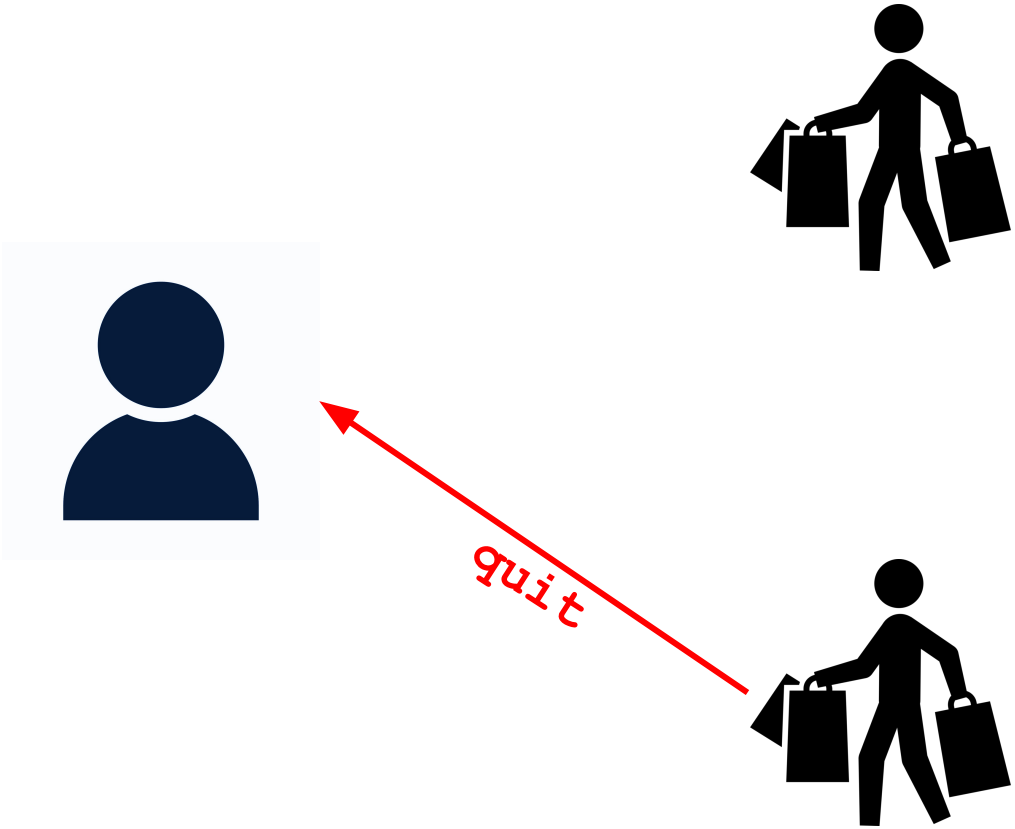


Example: two-bidder protocol

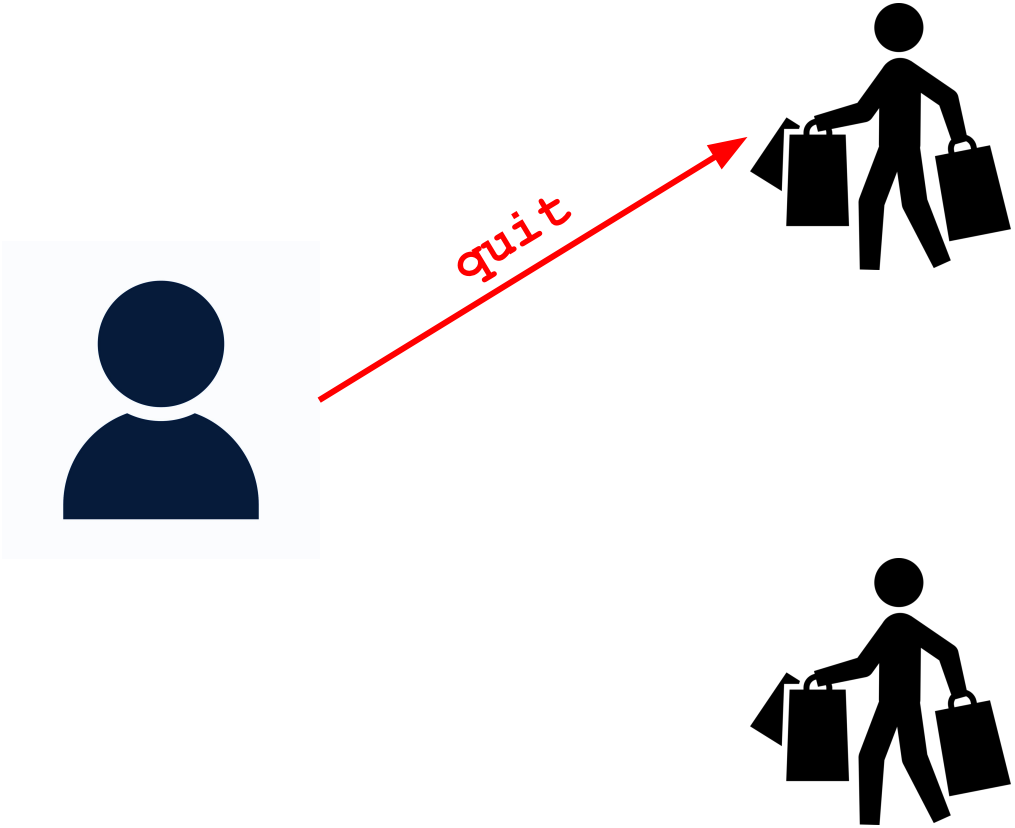
<z1,z2>

b2{b2>z2}

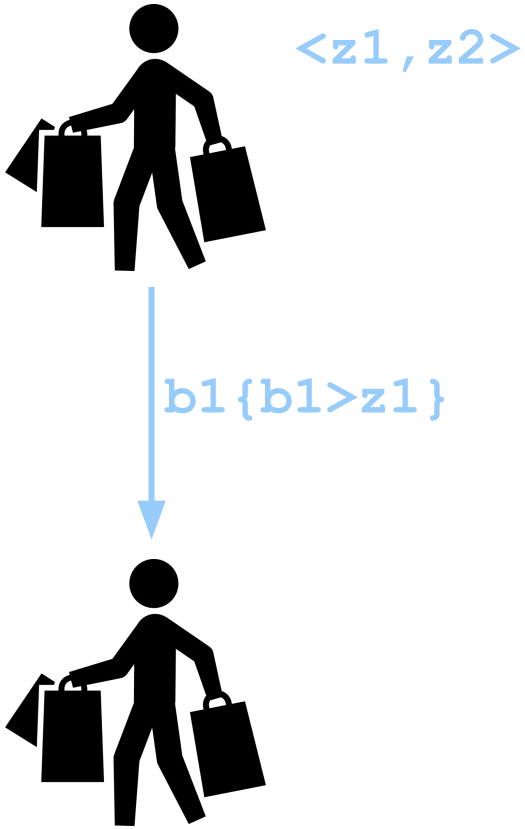
Example: two-bidder protocol



Example: two-bidder protocol



Example: two-bidder protocol

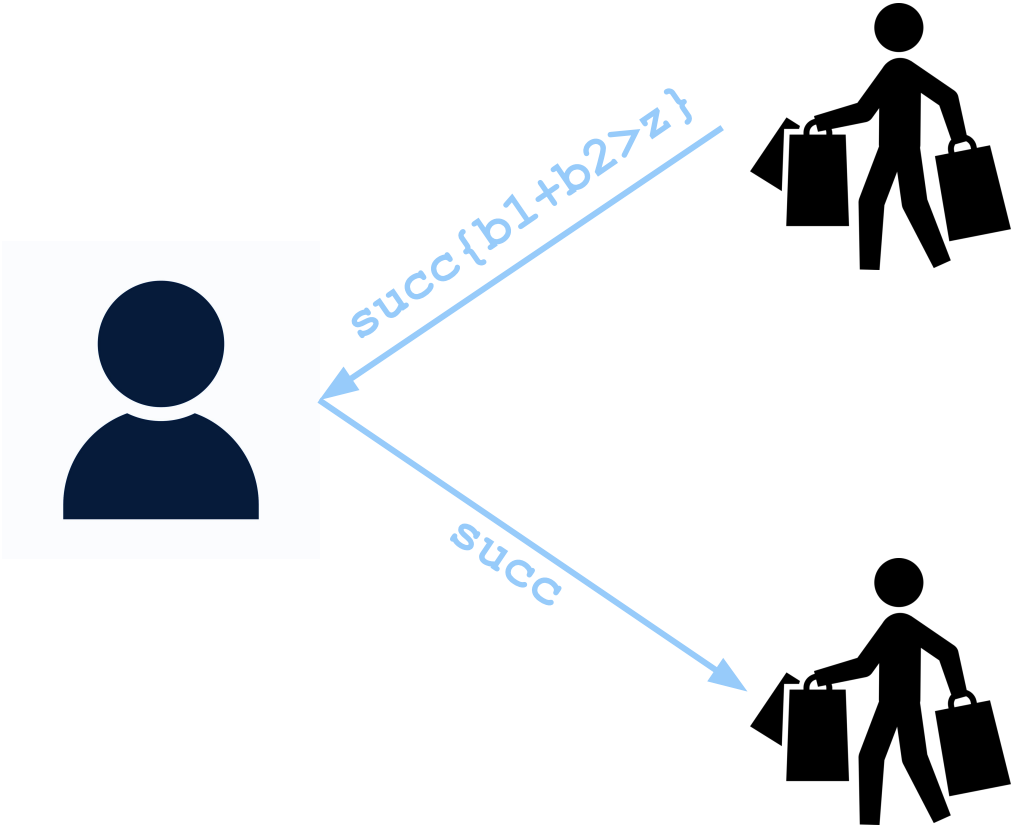


Example: two-bidder protocol

<z1,z2>

b2{b2>z2}

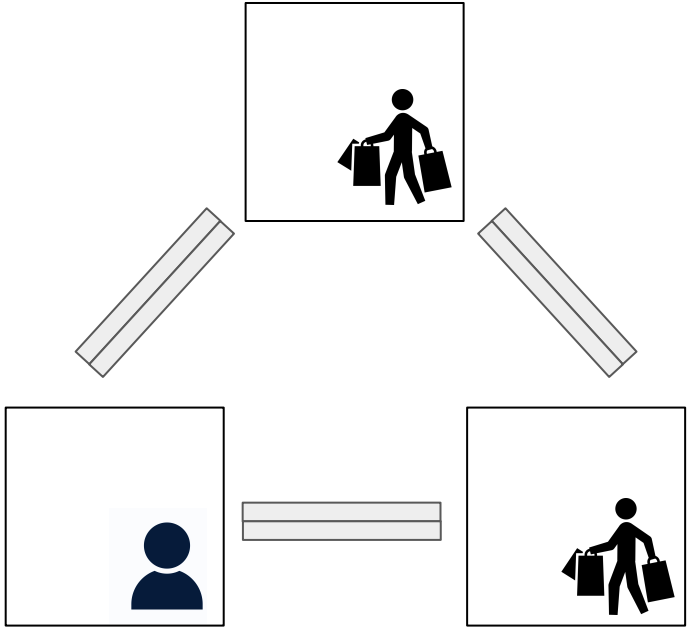
Example: two-bidder protocol



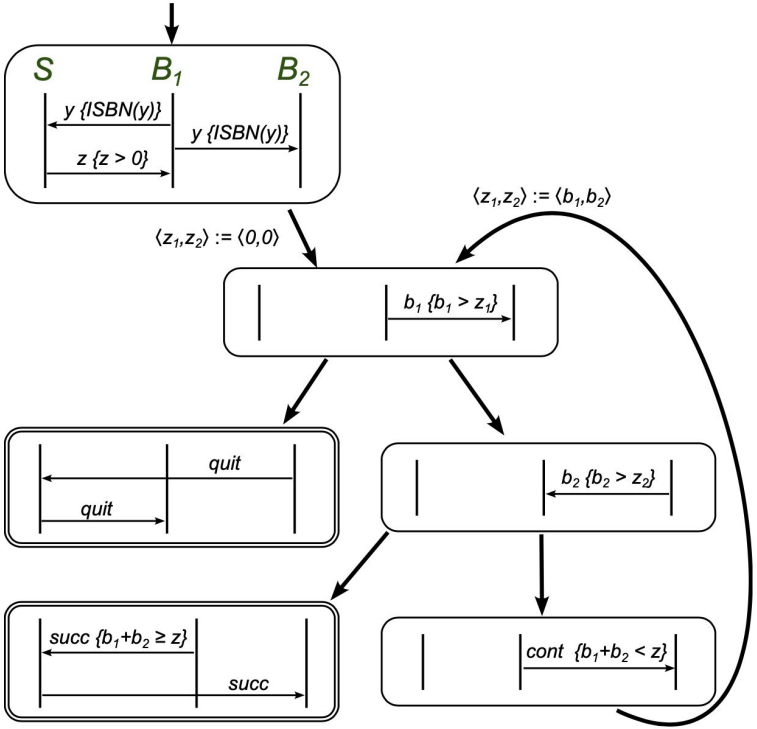
Overview

Asynchronous, message-passing programs are challenging to implement individually

- Communication errors e.g. orphan messages, unspecified receptions
- Deadlocks



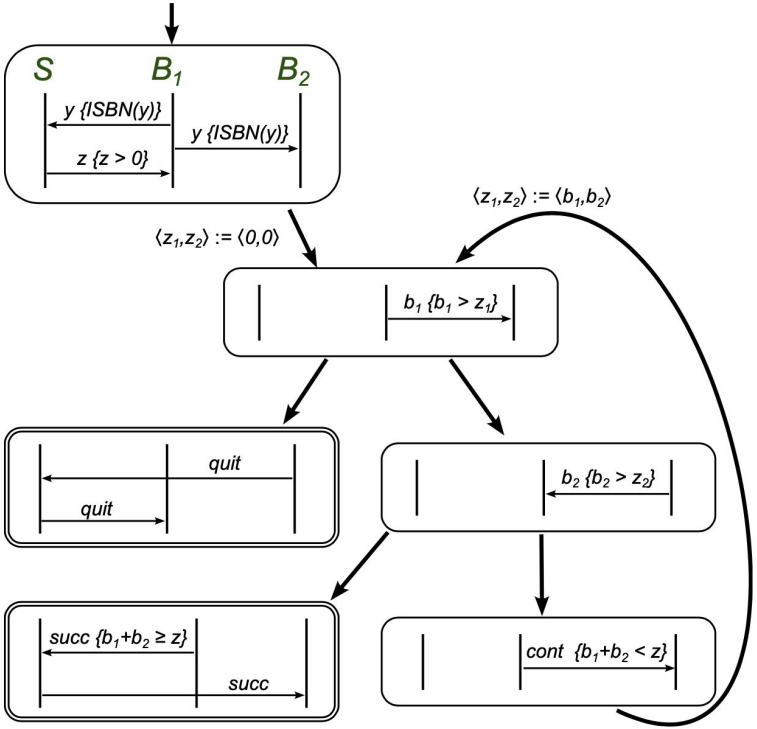
Overview



Global protocols are **synchronous** specifications of **all** participants' behaviors

- High-level message sequence charts [Mauw and Reniers 97]
- Session types [Honda et al. 08]
- Choreographic programming [Carbone and Montesi 13]

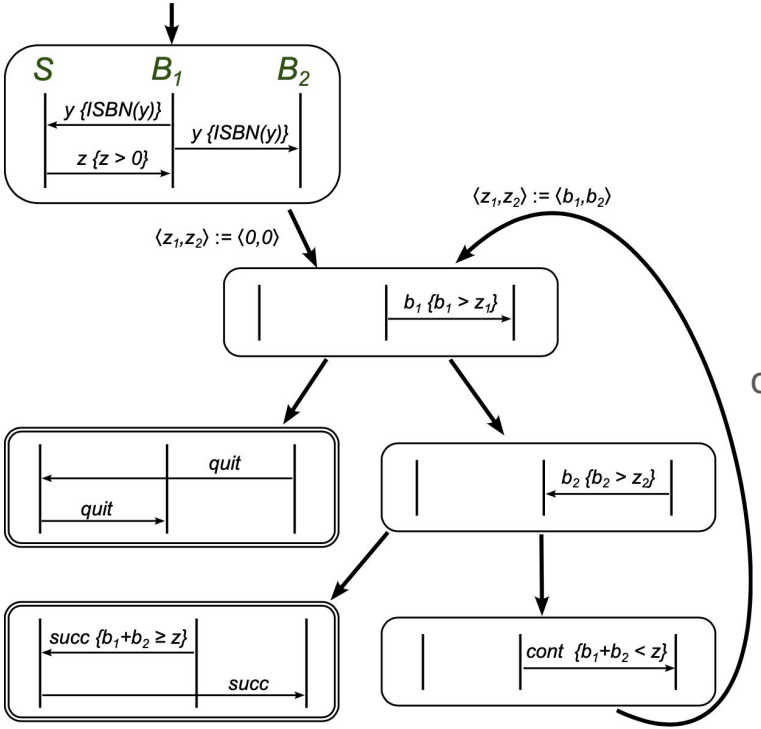
Overview

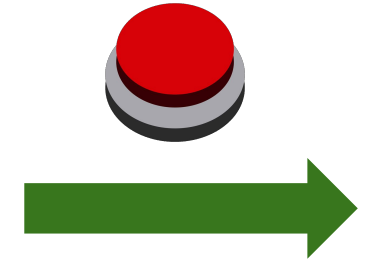


Applications:

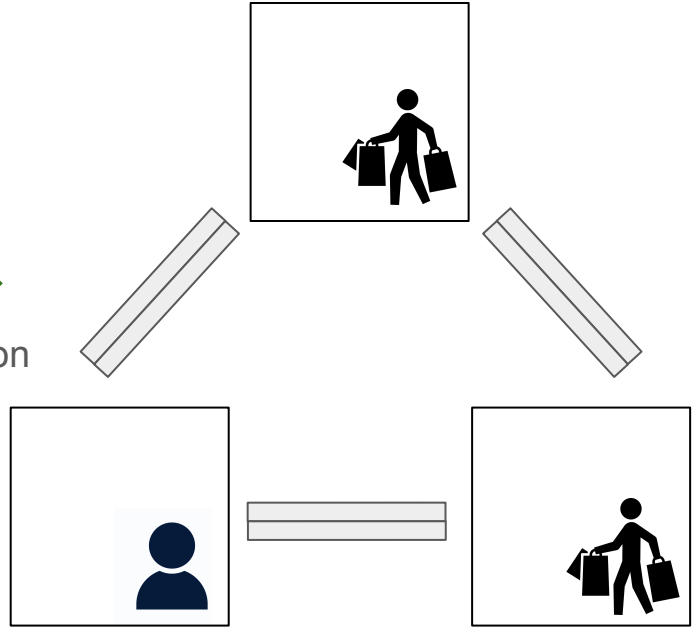
- ITU standard, UML, Web Service Choreography Description Language
- Cyber-physical systems
- Distributed architecture
- Cryptographic security protocols

Overview

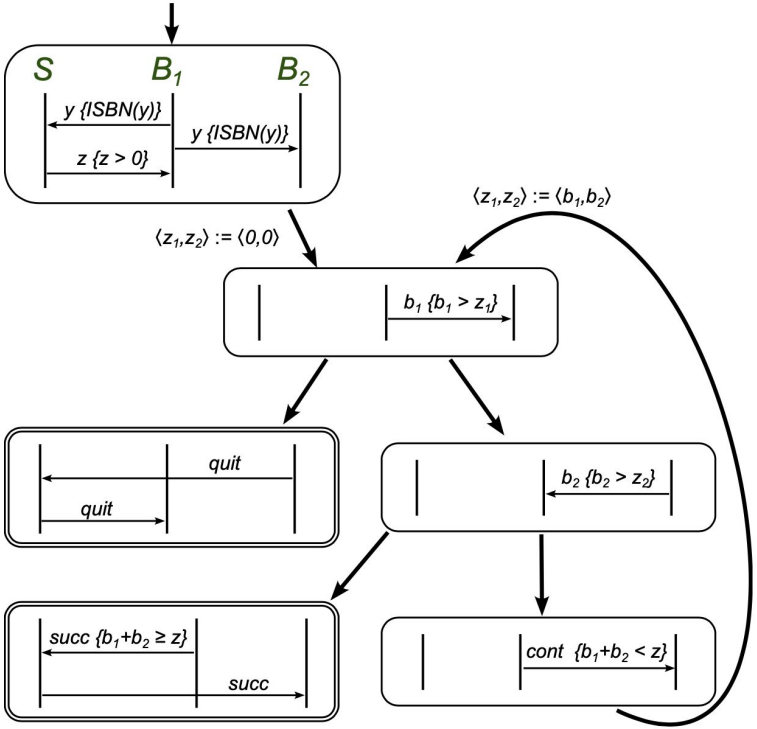




correct-by-construction synthesis

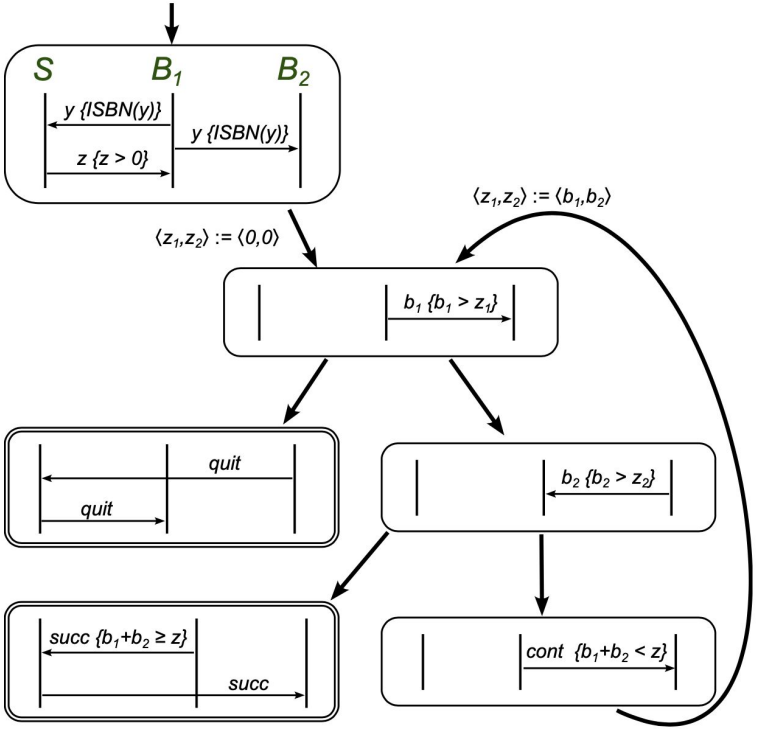


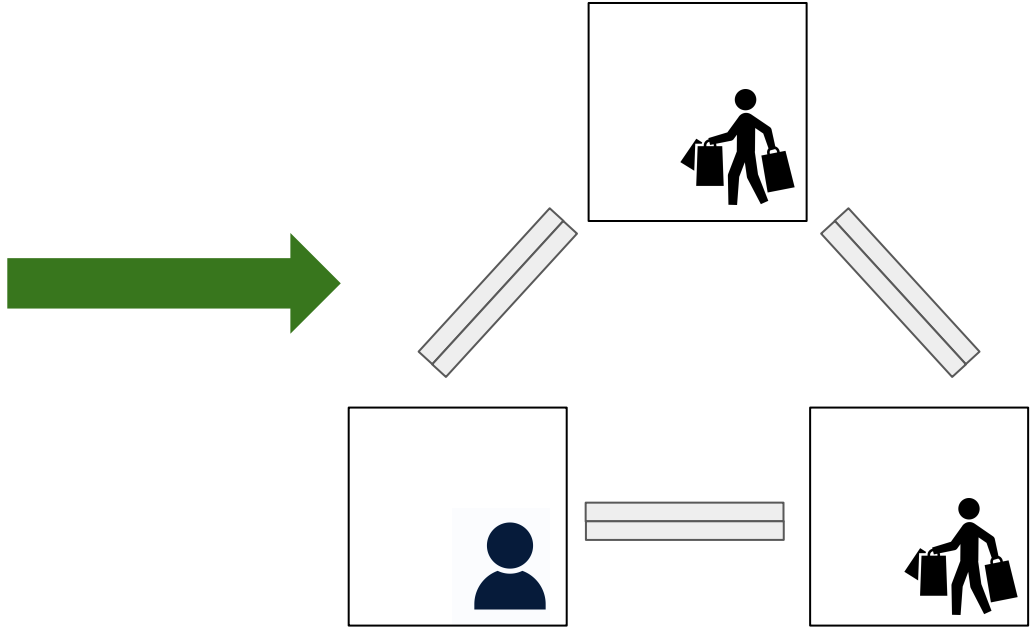
Overview


S
B1
B2
y {ISBN(y)}
y {ISBN(y)}
z {z > 0}
⟨z1,z2⟩ := ⟨0,0⟩
⟨z1,z2⟩ := ⟨b1,b2⟩
b1 {b1 > z1}
quit
quit
b2 {b2 > z2}
succ {b1+b2 ≥ z}
succ
cont {b1+b2 < z}

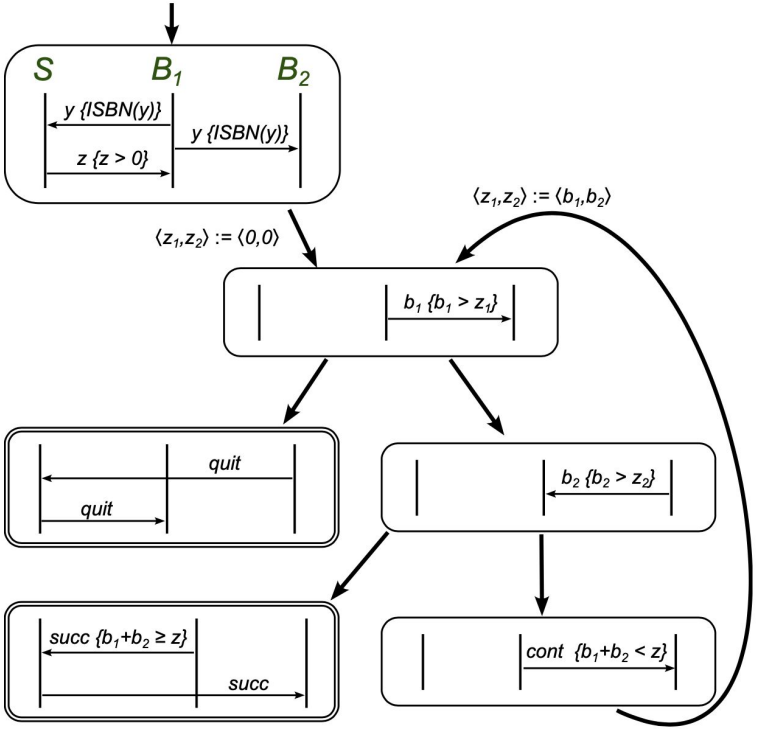
?
implementability

Overview

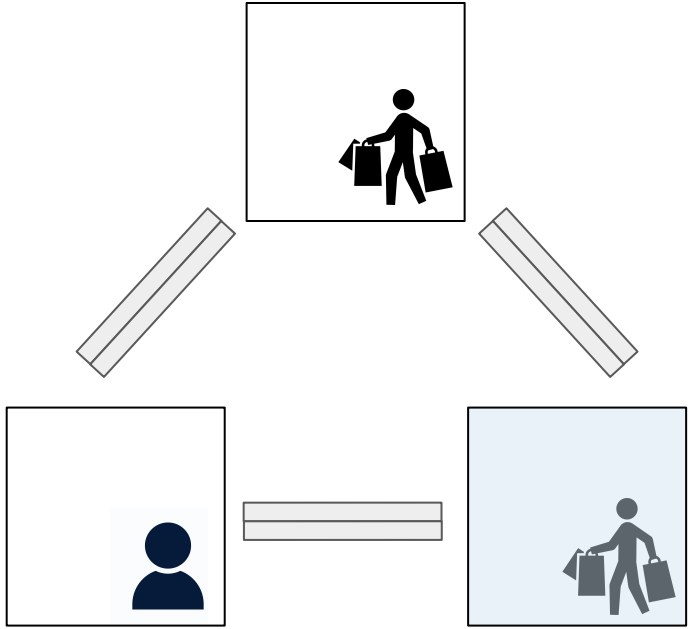

S
B1
B2
y {ISBN(y)}
y {ISBN(y)}
z {z > 0}
⟨z1,z2⟩ := ⟨0,0⟩
⟨z1,z2⟩ := ⟨b1,b2⟩
b1 {b1 > z1}
quit
quit
b2 {b2 > z2}
succ {b1+b2 ≥ z}
succ
cont {b1+b2 < z}



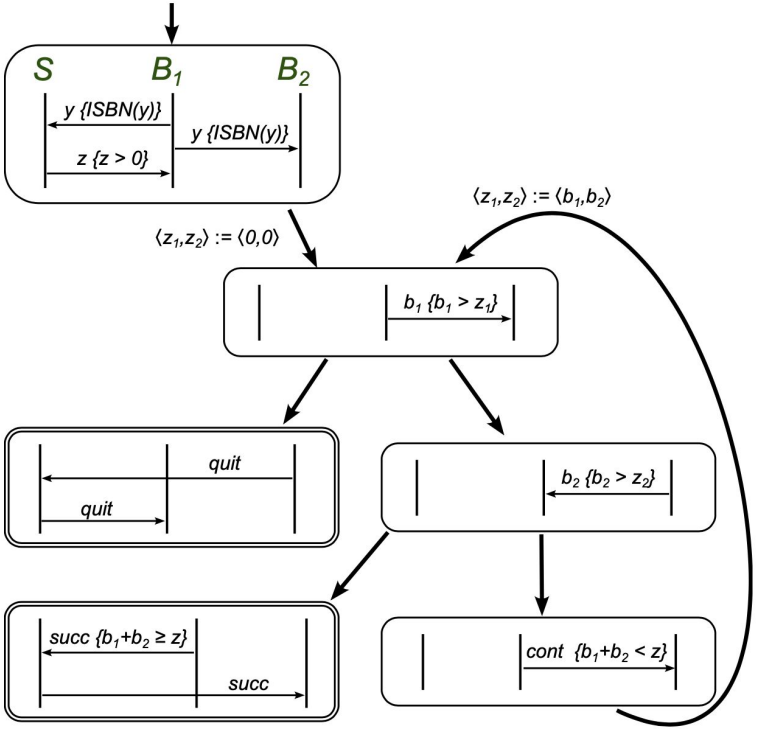
Overview



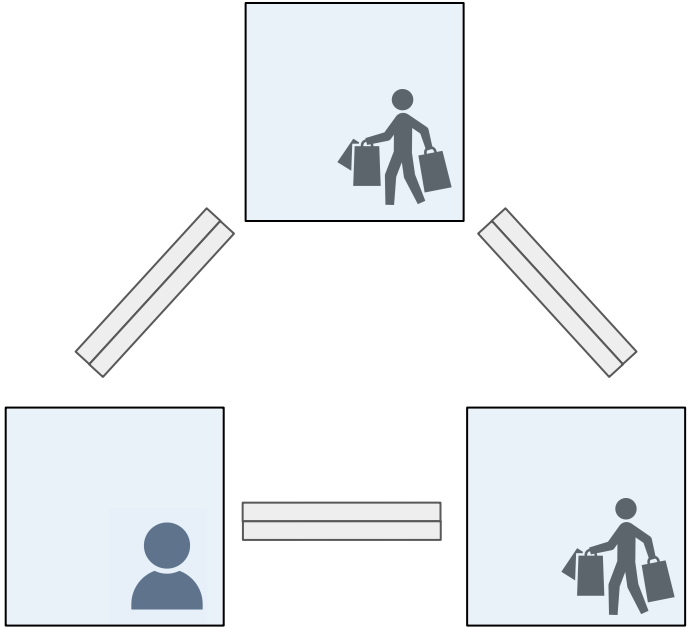
subtyping/
refinement



Overview



subtyping/
refinement



Contributions

- Implementability:
  - Network-parametric, precise characterization for infinite-state protocols (GCLTS)
  - Optimal decision procedures for finite fragments
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - Rocq mechanization
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Contributions

- [CAV'23] **Elaine Li**, **Felix Stutz**, Thomas Wies and Damien Zufferey. Complete Multiparty Session Type Projection with Automata.
- [ESOP'24] **Elaine Li**, Felix Stutz and Thomas Wies. Deciding Subtyping for Asynchronous Multiparty Sessions.
- [OOPSLA'25] **Elaine Li**, Felix Stutz, Thomas Wies and Damien Zufferey. Characterizing Implementability for Global Protocols with Infinite States and Data.
- [CAV'25] **Elaine Li**, Felix Stutz, Thomas Wies and Damien Zufferey. SPROUT: A Verifier for Symbolic Multiparty Protocols.
- [ITP'25] **Elaine Li** and Thomas Wies. Certified Implementability for Global Multiparty Protocols.
- **Elaine Li** and Thomas Wies. Implementability of Global Protocols Modulo Network Architectures. Under submission

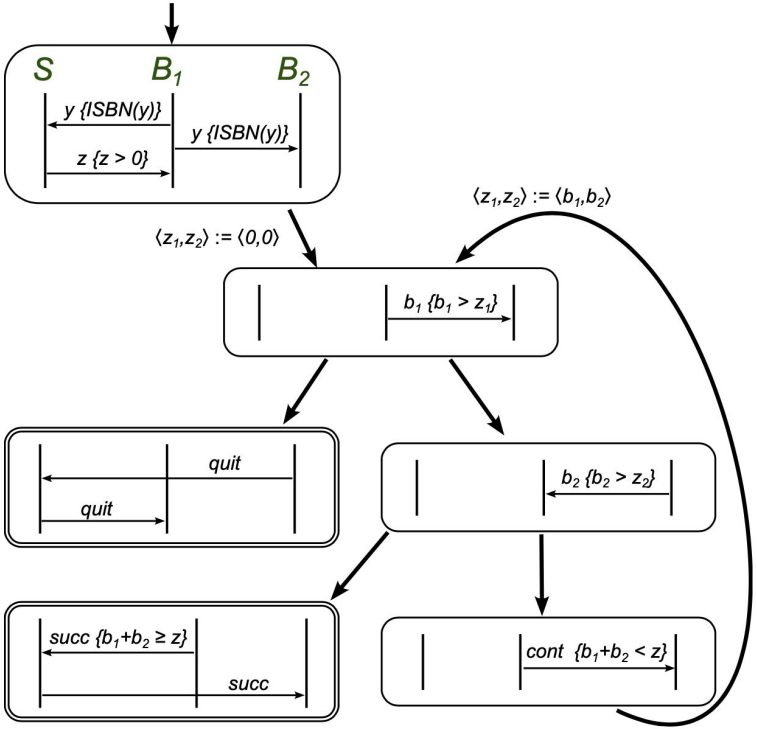
(New) contributions

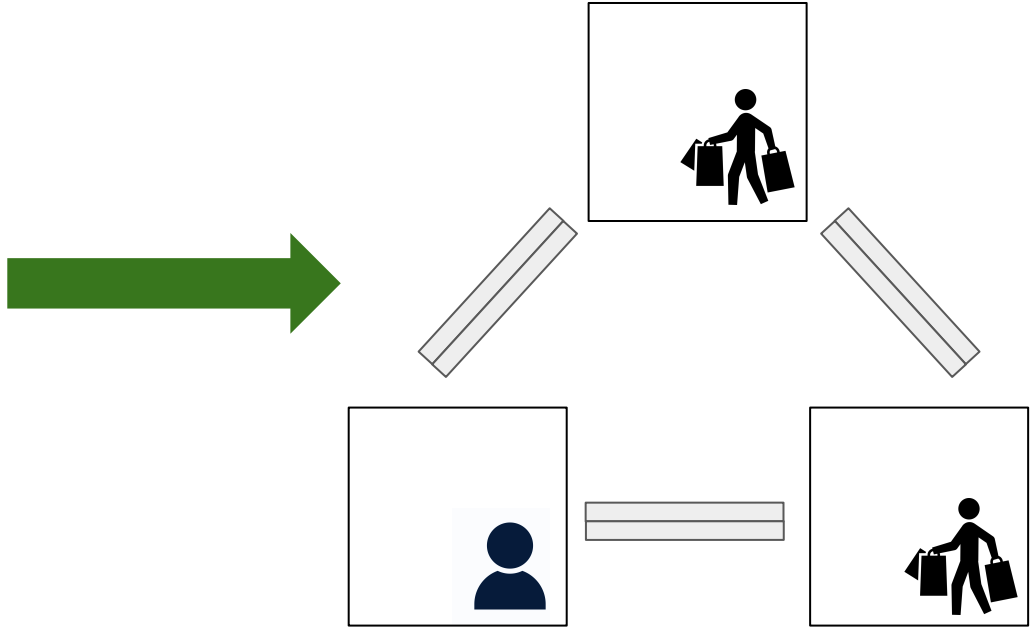
- [CAV'23] **Elaine Li**, **Felix Stutz**, Thomas Wies and Damien Zufferey. Complete Multiparty Session Type Projection with Automata.
- [ESOP'24] **Elaine Li**, Felix Stutz and Thomas Wies. Deciding Subtyping for Asynchronous Multiparty Sessions.
- [OOPSLA'25] **Elaine Li**, Felix Stutz, Thomas Wies and Damien Zufferey. Characterizing Implementability for Global Protocols with Infinite States and Data.
- [CAV'25] **Elaine Li**, Felix Stutz, Thomas Wies and Damien Zufferey. SPROUT: A Verifier for Symbolic Multiparty Protocols.
- [ITP'25] **Elaine Li** and Thomas Wies. Certified Implementability for Global Multiparty Protocols.
- **Elaine Li** and Thomas Wies. Implementability of Global Protocols Modulo Network Architectures. Under submission

Contributions

- Implementability:
  - **Network-parametric, precise characterization for infinite-state protocols (GCLTS)**
  - Optimal decision procedures for finite fragments
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - Rocq mechanization
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Overview

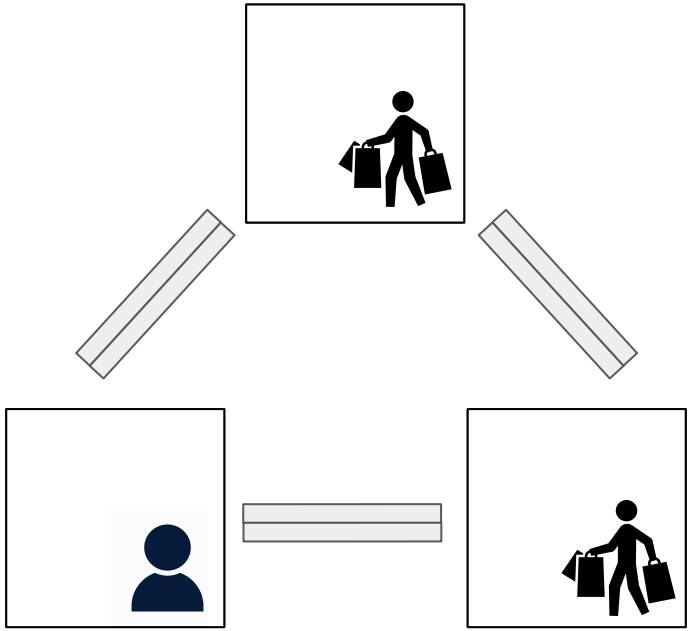

S
B1
B2
y {ISBN(y)}
y {ISBN(y)}
z {z > 0}
⟨z1,z2⟩ := ⟨0,0⟩
⟨z1,z2⟩ := ⟨b1,b2⟩
b1 {b1 > z1}
quit
quit
b2 {b2 > z2}
succ {b1+b2 ≥ z}
succ
cont {b1+b2 < z}



Implementation model

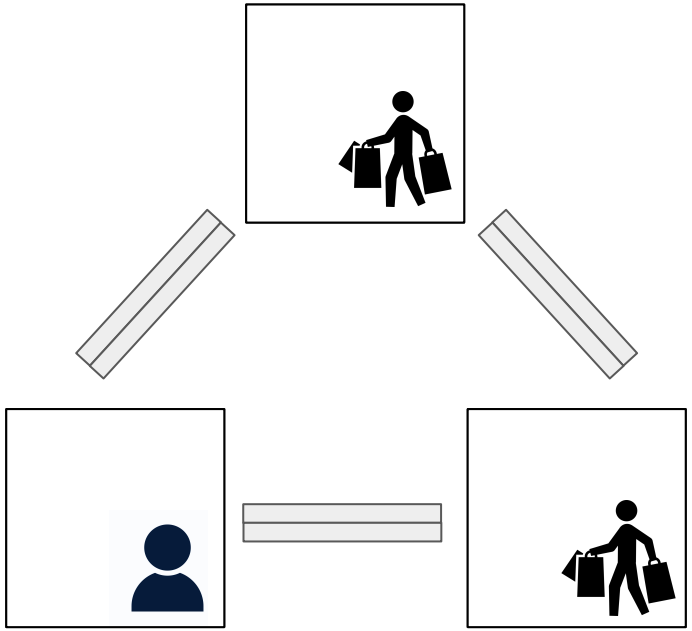
(controllable) **(non-controllable)**

Communicating Labeled Transition System (CLTS) = per participant LTS + network architecture A



Implementation model

Communicating Labeled Transition System (CLTS) = **per participant LTS** + network architecture A

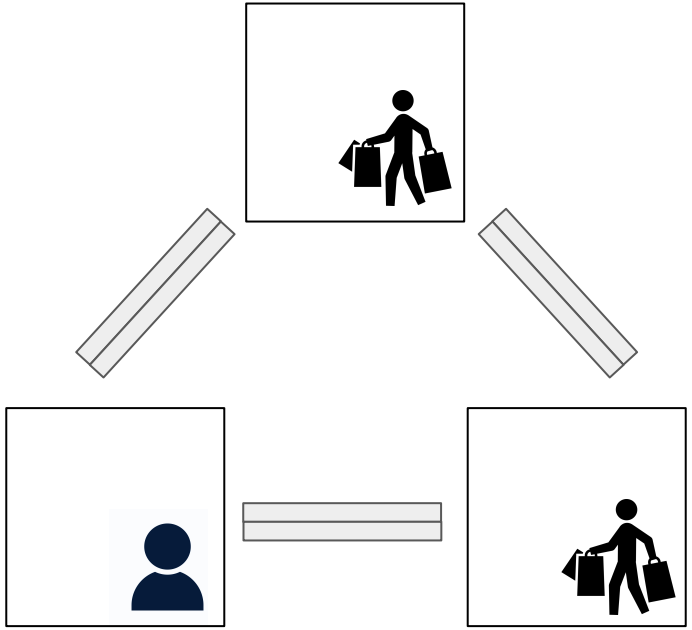


$b_1 \,!\, s : y \cdot b_1 \,!\, b_2 : y \cdot s \,?\, b_1 : y \cdot s \,!\, b_1 : z \cdot b_2 \,?\, b_1 : y$ ✓

$b_1 \,!\, s : y \cdot b_1 \,!\, b_2 : y \cdot s \,!\, b_1 : z \cdot s \,?\, b_1 : y \cdot b_2 \,?\, b_1 : y$ ✗

Implementation model

Communicating Labeled Transition System (CLTS) = per participant LTS + **network architecture $\mathbf{A}$**



Peer-to-peer, FIFO:

- Communicating state machines [Brand and Zafiropulo 83]
- Ubiquitous in theory and verification

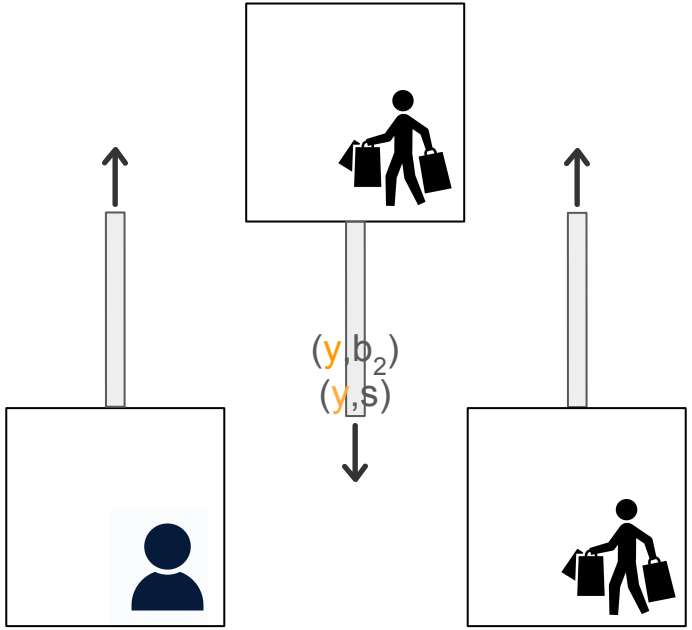
$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \cdot b_2 ? b_1 : y$ ✓

Implementation model

Communicating Labeled Transition System (CLTS) = per participant LTS + **network architecture A**



Senderbox:

- Work-stealing in parallel programming

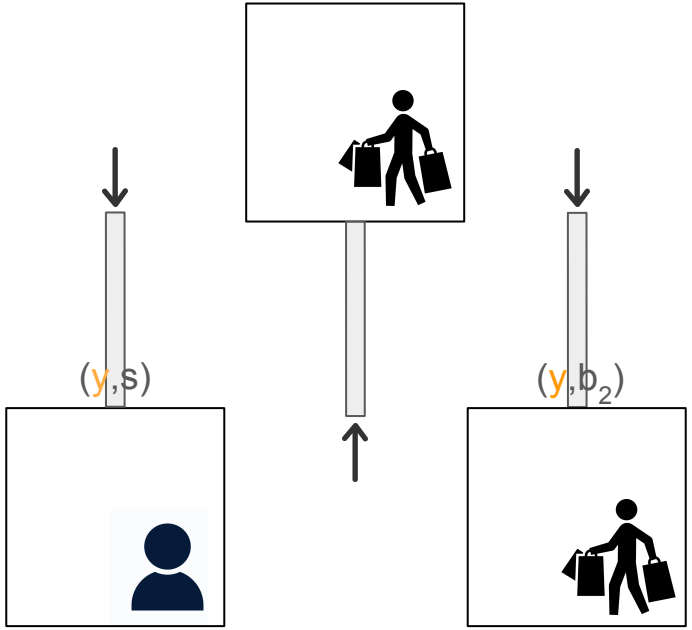
$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z$ ✗

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \cdot b_2 ? b_1 : y$ ✓

Implementation model

Communicating Labeled Transition System (CLTS) = per participant LTS + **network architecture $\mathbf{A}$**



Mailbox:

- Go, Erlang
- Synchronizability and realizability [Etienne and Lozes 17, Muscholl and Delpy 24]

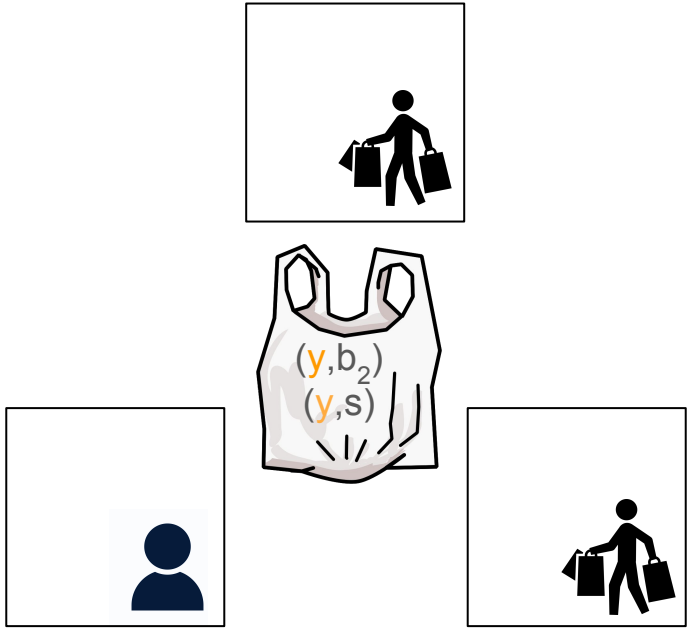
$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \cdot b_2 ? b_1 : y$ ✓

Implementation model

Communicating Labeled Transition System (CLTS) = per participant LTS + **network architecture $\mathbf{A}$**



Monobag:

- Consensus protocols

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z$ ✓

$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \cdot b_2 ? b_1 : y$ ✓

Network architecture $\mathbb{A}$

Communication topology	$\times$	Buffer type
n-n (peer-to-peer)		FIFO queue (box)
1-n (sender)		multiset (bag)
n-1 (mail)		
1-1 (mono)		

Global protocol semantics for $\mathbf{A}$

A word is **executable under $\mathbf{A}$** if it is a trace of some CLTS* under $\mathbf{A}$

Two words are **indistinguishable by $\mathbf{A}$** if any CLTS that executes one must execute the other

$$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z \quad \checkmark$$

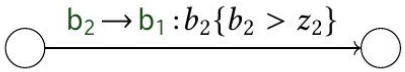
$$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \quad \checkmark$$

Global protocol semantics must be indistinguishability-closed

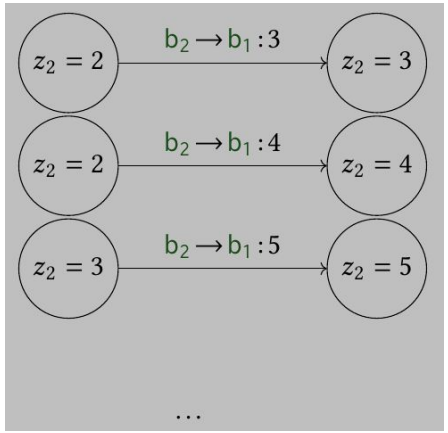
*non-deadlocking

Global protocol semantics

Symbolic transition

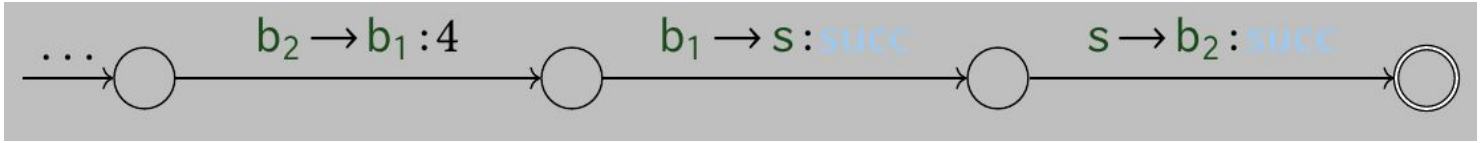


Concrete transition



Global protocol semantics

Concrete run



Synchronous

$$\ldots b_2 \to b_1 : 4 \cdot b_1 \to s : succ \cdot s \to b_2 : succ$$

Asynchronous

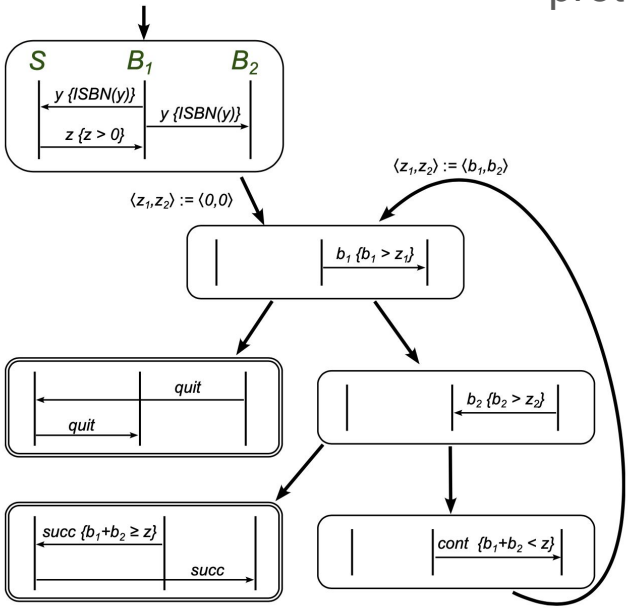
$$\ldots b_2 ! b_1 : 4 \cdot b_1 ? b_2 : 4 \cdot b_2 ! s : succ \cdot s ? b_2 : succ \cdot s ! b_2 : succ \cdot b_2 ? s : succ$$

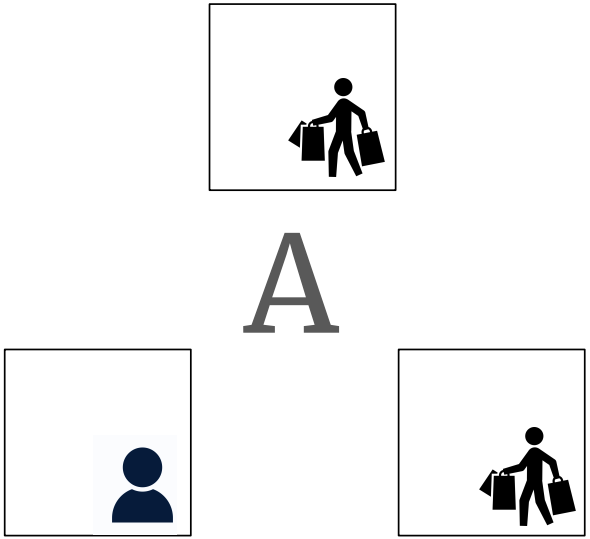
…and all words indistinguishable by $\mathbf{A}$

Network-parametric implementability

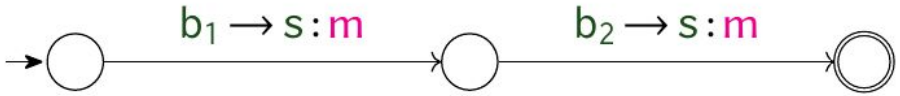
Given network architecture $\mathbb{A}$, exists an implementation satisfying

protocol fidelity + deadlock freedom?

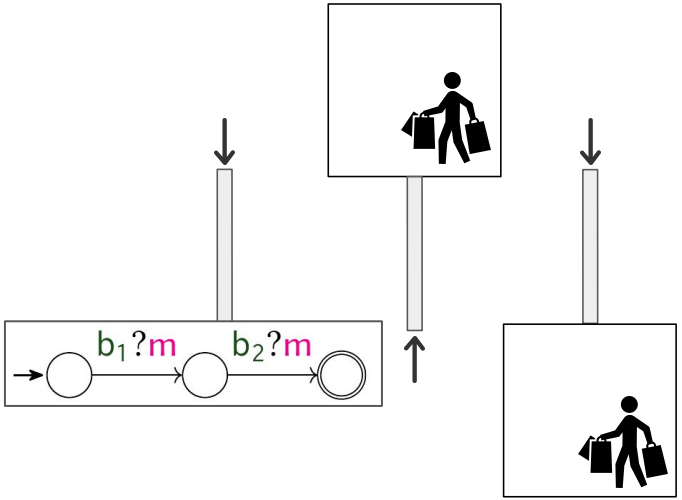




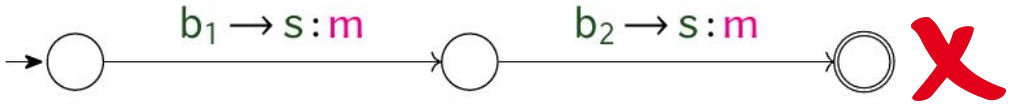
A non-implementable sequential specification



$$\mathcal{L}_{mailbox} = \{b_1 \,!\, s : m \cdot s \,?\, b_1 : m \cdot b_2 \,!\, s : m \cdot s \,?\, b_2 : m,$$
$$b_1 \,!\, s : m \cdot b_2 \,!\, s : m \cdot s \,?\, b_1 : m \cdot s \,?\, b_2 : m\}$$

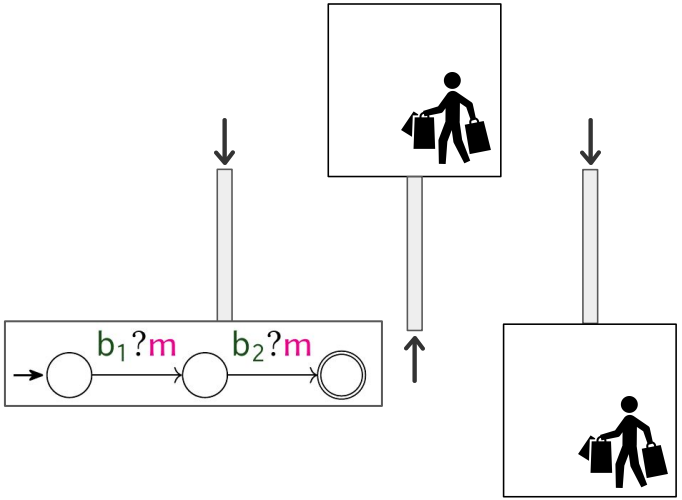


A non-implementable sequential specification



$$\mathcal{L}_{mailbox} = \{b_1\,!\,s{:}m \cdot s\,?\,b_1{:}m \cdot b_2\,!\,s{:}m \cdot s\,?\,b_2{:}m,$$
$$b_1\,!\,s{:}m \cdot b_2\,!\,s{:}m \cdot s\,?\,b_1{:}m \cdot s\,?\,b_2{:}m\}$$

$$b_2\,!\,s{:}m \cdot b_1\,!\,s{:}m$$



Global specification compatibility

A network architecture A is *global specification compatible* if no asynchronous words w ≡P w' that are executable under A can be distinguished by A.

$$b_1 \,!\, s : m \cdot b_2 \,!\, s : m \;\equiv P\; b_2 \,!\, s : m \cdot b_1 \,!\, s : m$$

peer-to-peer box, senderbox, ~~mailbox, monobox,~~
peer-to-peer bag, senderbag, mailbag, monobag

Coherence Conditions [Li et al. OOPSLA'25]

Theorem. A protocol is implementable if and only if it satisfies the Coherence Conditions.

In a nutshell:

From two locally indistinguishable global states, a participant can either:

- Send a message permissible from both states (Send Coherence)
- Receive a message distinguishing the two states (Receive Coherence)

but cannot choose between sending or receiving a message (No Mixed Choice).

Coherence Conditions [Li et al. OOPSLA'25]

Theorem. A protocol is implementable if and only if it satisfies the Coherence Conditions.

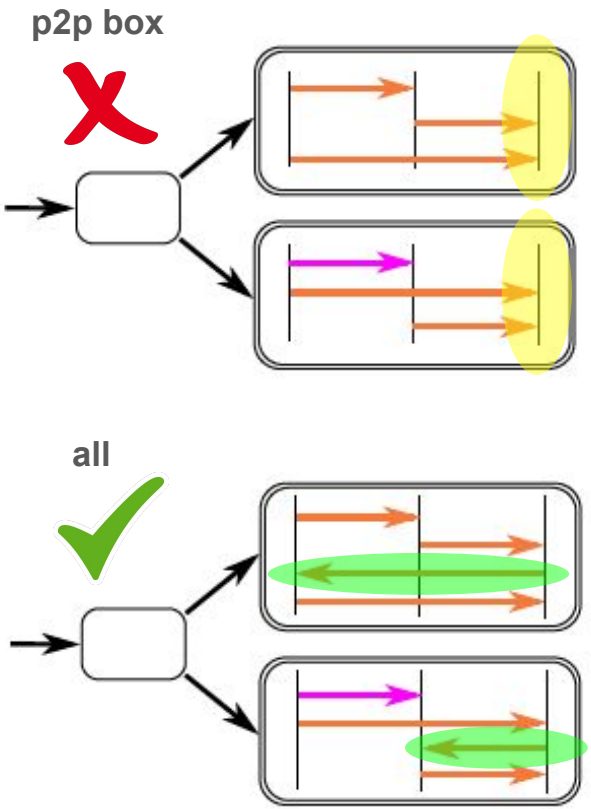
In a nutshell:

From two locally indistinguishable global states, a participant can either:

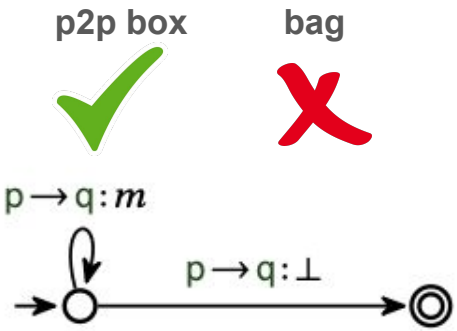
- Send a message permissible from both states (Send Coherence)
- **Receive a message distinguishing the two states (Receive Coherence),**

but cannot choose between sending or receiving a message (No Mixed Choice).

Receivability



Receivability



Network-parametricity goal

A <u>single</u> implementability characterization
and proof of preciseness
for all network architectures

Key steps towards network-parametricity

1) **Separation of concerns: controllable vs. uncontrollable**

L(A) = { w | *exists* CLTS T under network architecture A s.t. w is a trace of T }

Key steps towards network-parametricity

1) Separation of concerns: controllable vs. uncontrollable

~~L(A) = { w | *exists* CLTS T under network architecture A s.t. w is a trace of T }~~

operational

2) Operational ➡ algebraic definition of network architecture A:

correspondence

C(A) = { w | w is *channel-compliant* under A }

algebraic, inspired by [Majumdar et al. 22]

Key steps towards network-parametricity

1) **Separation of concerns: controllable vs. uncontrollable**

~~L(A) = { w | *exists* CLTS T under network architecture A s.t. w is a trace of T }~~

operational

2) **Operational ➡ algebraic definition of network architecture A:**

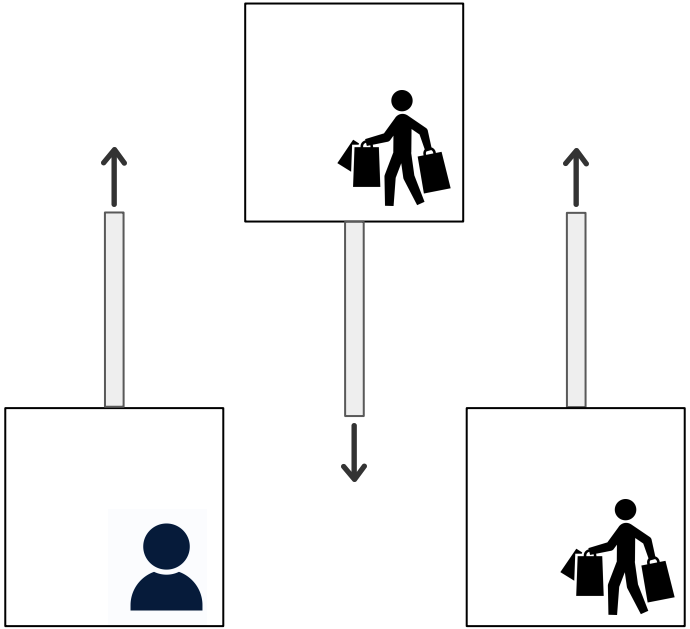
correspondence

C(A) = { w | w is *channel-compliant* under A }

algebraic, inspired by [Majumdar et al. 22]

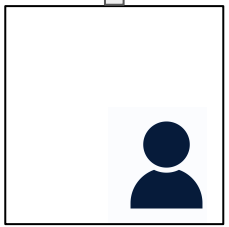
3) **Exploit algebraic definition to formulate *receivability* and network-parametric Coherence Conditions**

Key steps towards network-parametricity



Key steps towards network-parametricity

Definition 6.5 (Senderbox channel compliance). We say that w is *senderbox channel compliant* if for all prefixes $w' \leq w$, for all $p \in \mathcal{P}$, $w'\Downarrow_{_\triangleleft p?_} \leq w'\Downarrow_{p\triangleright_!_}$.



Key steps towards network-parametricity

Definition 6.3 (Peer-to-peer box channel compliance). We say that w is *p2p box channel compliant* if for all prefixes $w' \leq w$, for all $\mathsf{p} \neq \mathsf{q} \in \mathcal{P}$, $w'\Downarrow_{\mathsf{q}\triangleleft\mathsf{p}?_} \leq w'\Downarrow_{\mathsf{p}\triangleright\mathsf{q}!_}$.

Definition 6.4 (Peer-to-peer bag channel compliance). We say that w is *p2p bag channel compliant* if for all prefixes $w' \leq w$, for all $\mathsf{p} \neq \mathsf{q} \in \mathcal{P}$, $\{m \mid \mathsf{q} \triangleleft \mathsf{p}?m \in w'\} \subseteq \{m \mid \mathsf{p} \triangleright \mathsf{q}!m \in w'\}$.

Definition 6.5 (Senderbox channel compliance). We say that w is *senderbox channel compliant* if for all prefixes $w' \leq w$, for all $\mathsf{p} \in \mathcal{P}$, $w'\Downarrow_{\triangleleft\mathsf{p}?} \leq w'\Downarrow_{\mathsf{p}\triangleright\,!}$.

Definition 6.6 (Senderbag channel compliance). We say that w is *senderbag channel compliant* if for all prefixes $w' \leq w$, for all $\mathsf{p} \in \mathcal{P}$. $\{(m, \mathsf{q}) \mid \mathsf{q} \triangleleft \mathsf{p}?m \in w'\} \subseteq \{(m, \mathsf{q}) \mid \mathsf{p} \triangleright \mathsf{q}!m \in w'\}$.

Definition 6.7 (Mailbag channel compliance). We say that w is *mailbag channel compliant* if for all prefixes $w' \leq w$, for all $\mathsf{q} \in \mathcal{P}$. $\{(m, \mathsf{p}) \mid \mathsf{q} \triangleleft \mathsf{p}?m \in w'\} \subseteq \{(m, \mathsf{p}) \mid \mathsf{p} \triangleright \mathsf{q}!m \in w'\}$.

Definition 6.8 (Monobag channel compliance). We say that w is *monobag channel compliant* if for all prefixes $w' \leq w$, $\{(m, \mathsf{p}, \mathsf{q}) \mid \mathsf{q} \triangleleft \mathsf{p}?m \in w'\} \subseteq \{(m, \mathsf{p}, \mathsf{q}) \mid \mathsf{p} \triangleright \mathsf{q}!m \in w'\}$.

Key steps towards network-parametricity

<u>Channel compliance facts</u>

"Messages are sent before they are received"

"Synchronized traces can always be executed"

"Messages can always be sent"

etc.

Key steps towards network-parametricity

1) **Separation of concerns: controllable vs. uncontrollable**

~~L(A) = { w | *exists* CLTS T under network architecture A s.t. w is a trace of T }~~

operational

2) **Operational ➡ algebraic definition of network architecture A:**

correspondence

C(A) = { w | w is *channel-compliant* under A }

algebraic, inspired by [Majumdar et al. 22]

3) **Exploit algebraic definition to formulate *receivability* and network-parametric Coherence Conditions**
4) **Reason about all network architectures together in terms of their abstract properties**

Contributions

- Implementability:
  - Network-parametric, precise characterization for infinite-state protocols (GCLTS)
  - **Optimal decision procedures for finite fragments**
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - Rocq mechanization
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Algorithm for finite protocols

Algorithm 1 Check *CC* for finite protocols

▹ *Let LTS* $\mathcal{S} = (S, \Gamma, T, s_0, F)$

▹ *Checking Send Coherence*

for $s_1 \xrightarrow{\mathsf{p}\to\mathsf{q}:m} s_2 \in T$ **do**

  for $s \neq s_1 \in S$ **do**

    if $\mathcal{L}(S, \Gamma_\mathsf{p} \uplus \{\varepsilon\}, T_\mathsf{p}, s_0, \{s\}) \cap \mathcal{L}(S, \Gamma_\mathsf{p} \uplus \{\varepsilon\}, T_\mathsf{p}, s_0, \{s_1\}) \neq \emptyset$ **then**

      $b \leftarrow \bot$

      for $s_3 \xrightarrow{\mathsf{p}\to\mathsf{q}:m} s_4 \in T$ **do** $b \leftarrow b \vee \left(s \underset{\mathsf{p}}{\overset{\varepsilon}{\Rightarrow}}{}^* s_3\right)$

      if $\neg b$ **then return** $\bot$

▹ *Checking Receive Coherence*

for $s_1 \xrightarrow{\mathsf{p}\to\mathsf{q}:m} s_2, s_3 \xrightarrow{\mathsf{r}\to\mathsf{q}:m} s_4 \in T, s_1 \neq s_2, \mathsf{p} \neq \mathsf{r}$ **do**

  if $\mathcal{L}(S, \Gamma_\mathsf{q} \uplus \{\varepsilon\}, T_\mathsf{q}, s_0, \{s_1\}) \cap \mathcal{L}(S, \Gamma_\mathsf{q} \uplus \{\varepsilon\}, T_\mathsf{q}, s_0, \{s_3\}) \neq \emptyset$ **then**

    if $\mathsf{avail}_{\mathsf{p},\mathsf{q},\{\mathsf{q}\}}(m, s_4)$ **then return** $\bot$

▹ *Checking No Mixed Choice*

for $s_1 \xrightarrow{\mathsf{p}\to\mathsf{q}:m} s_2, s_3 \xrightarrow{\mathsf{r}\to\mathsf{p}:m} s_4 \in T, s_1 \neq s_2$ **do**

  if $\mathcal{L}(S, \Gamma_\mathsf{q} \uplus \{\varepsilon\}, T_\mathsf{q}, s_0, \{s_1\}) \cap \mathcal{L}(S, \Gamma_\mathsf{q} \uplus \{\varepsilon\}, T_\mathsf{q}, s_0, \{s_3\}) \neq \emptyset$ **then return** $\bot$

return $\top$

Complexity for finite fragments (p2p box)

Fragment	**Complexity**
Symbolic, boolean	PSPACE-complete
Non-symbolic	co-NP-complete
Non-symbolic multiparty session types	co-NP-complete

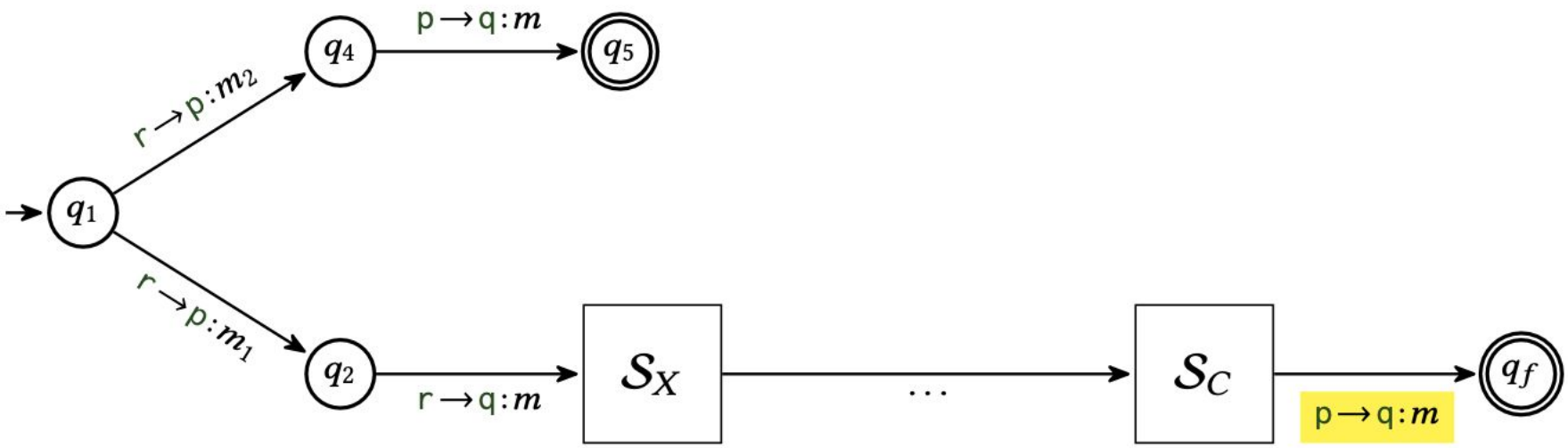
Complexity for finite fragments under A

Fragment	Complexity
Symbolic, boolean	PSPACE-complete
Non-symbolic	co-NP-complete
Non-symbolic multiparty session types	co-NP-complete

Complexity

3-SAT instance: set of variables X, set of clauses C

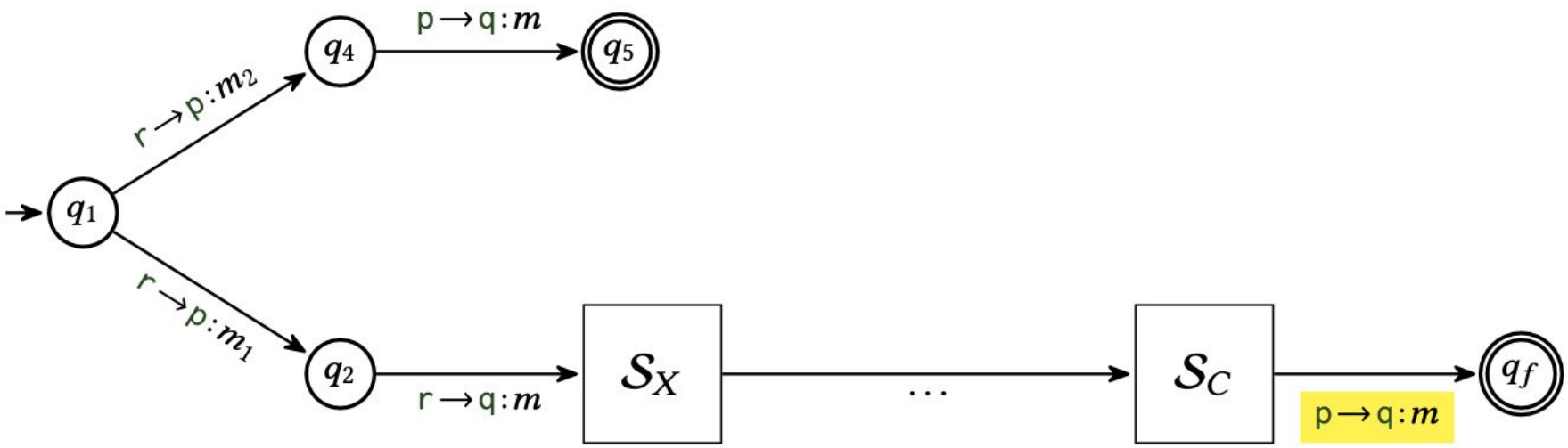
S is implementable iff 3-SAT instance is satisfiable



Complexity

3-SAT instance: set of variables X, set of clauses C

~~S is implementable~~ m is receivable iff 3-SAT instance is satisfiable



Contributions

- Implementability:
  - Network-parametric, precise characterization for infinite-state protocols (GCLTS)
  - Optimal decision procedures for finite fragments
  - **Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT**
  - Rocq mechanization
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Algorithm for symbolic protocols

$$\begin{aligned}
&\mathsf{unreach}^{\varepsilon}_{\mathsf{p},\mathsf{q}}(s, \boldsymbol{r}, x) :=_{\nu} \\
&\wedge \Big(\bigwedge_{(s,\mathsf{p}\to\mathsf{q}:y\{\varphi\},s')\in\Delta} \neg\varphi[x/y] \Big) \\
&\wedge \Big(\bigwedge_{\substack{(s,\mathsf{q}\to\mathsf{t}:y\{\varphi\},s')\in\Delta \\ \mathsf{p}\neq\mathsf{q}\wedge\mathsf{p}\neq\mathsf{t}}} \forall y.\, \varphi \Rightarrow \mathsf{unreach}^{\varepsilon}_{\mathsf{p},\mathsf{q}}(s', \boldsymbol{r}', x)\Big) \ .
\end{aligned}$$

sound and complete
μCLP reduction
(fixpoint logic modulo first order theories)

Coherence Conditions → **Symbolic Coherence Conditions**

$$\begin{aligned}
&\mathsf{prodreach}_{\mathsf{p}}(s_1', \boldsymbol{r}_1', s_2', \boldsymbol{r}_2') :=_{\mu} \\
&\vee (s_1' = s_0 \wedge s_2' = s_0) \\
&\vee \Big(\bigvee_{\substack{(s_1,\mathsf{r}\to\mathsf{s}:x_1\{\varphi_1\},s_1')\in\Delta_1 \\ (s_2,\mathsf{r}\to\mathsf{s}:x_2\{\varphi_2\},s_2')\in\Delta_2 \\ \mathsf{p}=\mathsf{r}\vee\mathsf{p}=\mathsf{s}}} \exists x_1 x_2.\, \mathsf{prodreach}_{\mathsf{p}}(s_1, \boldsymbol{r}_1, s_2, \boldsymbol{r}_2) \wedge \varphi_1 \wedge \varphi_2 \wedge x_1 = x_2\Big) \\
&\vee \Big(\bigvee_{\substack{(s_1,\mathsf{r}\to\mathsf{s}:x_1\{\varphi_1\},s_1')\in\Delta_1 \\ \mathsf{p}\neq\mathsf{r}\wedge\mathsf{p}\neq\mathsf{s}}} \exists x_1.\, \mathsf{prodreach}_{\mathsf{p}}(s_1, \boldsymbol{r}_1, s_2', \boldsymbol{r}_2') \wedge \varphi_1\Big) \\
&\vee \Big(\bigvee_{\substack{(s_2,\mathsf{r}\to\mathsf{s}:x_2\{\varphi_2\},s_2')\in\Delta_2 \\ \mathsf{p}\neq\mathsf{r}\wedge\mathsf{p}\neq\mathsf{s}}} \exists x_2.\, \mathsf{prodreach}_{\mathsf{p}}(s_1', \boldsymbol{r}_1', s_2, \boldsymbol{r}_2) \wedge \varphi_2\Big) \ .
\end{aligned}$$

$$\begin{aligned}
&\mathsf{avail}_{\mathsf{p},\mathsf{q},\mathcal{B}}(x_1, s, \boldsymbol{r}) :=_{\mu} \\
&\bigvee_{\substack{(s,\mathsf{r}\to\mathsf{t}:x\{\varphi\},s')\in\Delta \\ \mathsf{r}\in\mathcal{B} \\ \mathsf{r}\neq\mathsf{p}\vee\mathsf{t}\neq\mathsf{q}}} \exists x.\, \mathsf{avail}_{\mathsf{p},\mathsf{q},\mathcal{B}\cup\{\mathsf{t}\}}(x_1, s', \boldsymbol{r}') \wedge \varphi \\
&\bigvee_{\substack{(s,\mathsf{r}\to\mathsf{t}:x\{\varphi\},s')\in\Delta \\ \mathsf{r}\notin\mathcal{B} \\ \mathsf{r}\neq\mathsf{p}\vee\mathsf{t}\neq\mathsf{q}}} \exists x.\, \mathsf{avail}_{\mathsf{p},\mathsf{q},\mathcal{B}}(x_1, s', \boldsymbol{r}') \wedge \varphi \\
&\bigvee_{\substack{(s,\mathsf{p}\to\mathsf{q}:x\{\varphi\},s')\in\Delta \\ \mathsf{p}\notin\mathcal{B}}} \varphi[x_1/x] \ .
\end{aligned}$$

SPROUT [Li et al. CAV'25]

CAV Artifact Evaluation ★ Available

CAV Artifact Evaluation ★★★ Reusable

Input

```
Initial state: (0)
Initial register assignments: ry=0, rc=0, rz1=0, rz2=0
(0) B1->S:y{(y>987000000000/\y<9880000000000)/\ry'=y} (1)
(1) B1->B2:y{y=ry} (2)
(2) S->B1:z{z>0/\rc'=z} (3)
(3) B1->B2:b1{b1>rz1/\rz1'=b1} (4)
(4) B2->S:quit{quit=0} (5)
(5) S->B1:quit{quit=0} (6)
(4) B2->B1:b2{b2>rz2/\b2<rc/\rz2'=b2} (7)
(7) B1->S:succ{succ=1/\rz1+rz2>=rc} (8)
(8) S->B2:succ{succ=1} (6)
(7) B1->B2:cont{cont=2/\rz1+rz2<rc} (3)
Final states: (6)
```

↓

Symbolic Coherence Conditions [OOPSLA'25]

↓

MuVal [Unno et al. 23]

✓ ✗

Non-implementable two-bidder protocol

<z1,z2>

b2{b2>z2 /\ z2<z}

Reporting implementability violations

```
Initial state: (0)
Initial register assignments: ry=0, rc=0, rz1=0, rz2=0
(0) B1->S:y{(y>987000000000/\y<9880000000000)/\ry'=y} (1)
(1) B1->B2:y{y=ry} (2)
(2) S->B1:z{z>0/\rc'=z} (3)
(3) B1->B2:b1{b1>rz1/\rz1'=b1} (4)
(4) B2->S:quit{quit=0} (5)
(5) S->B1:quit{quit=0} (6)
(4) B2->B1:b2{b2>rz2/\b2<rc/\rz2'=b2} (7)
(7) B1->S:succ{succ=1/\rz1+rz2>=rc} (8)
(8) S->B2:succ{succ=1} (6)
(7) B1->B2:cont{cont=2/\rz1+rz2<rc} (3)
Final states: (6)
```

B2's transition
from (4) to (7) violates
Symbolic Send Coherence

Expressivity + efficiency

[Zhou et al. 20]

Example	$\lvert\mathcal{P}\rvert$	Impl.	SPROUT	Time	SESSION*	Time
Calculator	2	✓	✓	0.6s	N/A	2.0s
Fibonacci	2	✓	✓	0.5s	✓	1.8s
HigherLower	3	✓	✓	15.2s	✓	3.9s
HTTP	2	✓	✓	0.4s	✓	1.9s
Negotiation	2	✓	✓	1.0s	✓	1.9s
OnlineWallet	3	✓	✓	9.4s	✓	3.3s
SH	3	✓	✓	237.1s	✓	5.6s
Ticket	2	✓	✓	0.6s	✓	1.9s
TravelAgency	2	✓	✓	9.2s	✓	3.1s
TwoBuyer	3	✓	✓	3.8s	✓	2.8s
DoubleBuffering	3	✓	✓	1.5s	✓	2.3s
OAuth	3	✓	✓	6.2s	✓	2.3s
PlusMinus	3	✓	✓	5.2s	×	2.1s
RingMax	7	✓	✓	3.7s	✓	4.7s
SimpleAuth	2	✓	✓	0.5s	✓	2.0s
TravelAgency2	2	✓	✓	1.7s	✓	1.8s

Expressivity + efficiency

[Zhou et al. 20]

Example	$\lvert\mathcal{P}\rvert$	Impl.	SPROUT	Time	SESSION*	Time
Calculator	2	✓	✓	0.6s	N/A	2.0s
Fibonacci	2	✓	✓	0.5s	✓	1.8s
HigherLower	3	✓	✓	15.2s	✓	3.9s
HTTP	2	✓	✓	0.4s	✓	1.9s
Negotiation	2	✓	✓	1.0s	✓	1.9s
OnlineWallet	3	✓	✓	9.4s	✓	3.3s
SH	3	✓	✓	237.1s	✓	5.6s
Ticket	2	✓	✓	0.6s	✓	1.9s
TravelAgency	2	✓	✓	9.2s	✓	3.1s
TwoBuyer	3	✓	✓	3.8s	✓	2.8s
DoubleBuffering	3	✓	✓	1.5s	✓	2.3s
OAuth	3	✓	✓	6.2s	✓	2.3s
PlusMinus	3	✓	✓	5.2s	×	2.1s
RingMax	7	✓	✓	3.7s	✓	4.7s
SimpleAuth	2	✓	✓	0.5s	✓	2.0s
TravelAgency2	2	✓	✓	1.7s	✓	1.8s

Multiparty session types literature contains only positive examples!

Precision

[Zhou et al. 20]

Example	$\lvert\mathcal{P}\rvert$	Impl.	SPROUT	Time	SESSION*	Time
send-validity-yes	4	✓	✓	1.9s	×	2.1s
send-validity-no	4	×	×	1.9s	×	2.1s
receive-validity-yes	3	✓	✓	5.1s	×	2.3s
receive-validity-no	3	×	×	3.6s	×	2.0s
symbolic-two-bidder-yes	3	✓	✓	27.4s	×	2.0s
symbolic-two-bidder-no1	3	×	×	30.0s	×	2.1s
figure12-yes	3	✓	✓	2.0s	✓	2.0s
figure12-no	3	×	×	3.0s	✓	3.0s
symbolic-send-validity-yes	4	✓	✓	6.5s	×	2.5s
symbolic-send-validity-no	4	×	×	5.3s	×	2.6s
symbolic-receive-validity-yes	3	✓	✓	6.6s	×	2.8s
symbolic-receive-validity-no	3	×	×	7.6s	×	2.8s
fwd-auth-yes	3	✓	✓	10.3s	×	2.3s
fwd-auth-no	3	×	?	T/O	×	2.2s
symbolic-two-bidder-no2	3	×	×	23.9s	×	2.8s
higher-lower-ultimate	3	✓	✓	11.1s	×	2.4s
higher-lower-winning	3	✓	?	T/O	✓	229.8s
higher-lower-no	3	×	×	7.3s	N/A	2.2s
higher-lower-encrypt-yes	4	✓	✓	9.3s	×	2.3s
higher-lower-encrypt-no	4	×	×	177.3s	×	2.4s
higher-lower-mixed	3	×	×	19.3s	×	2.3s

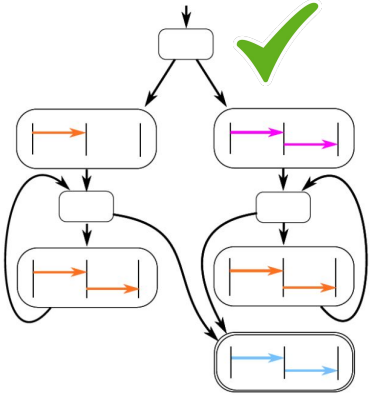
Precision

[Zhou et al. 20]

Example	$\lvert\mathcal{P}\rvert$	Impl.	SPROUT	Time	SESSION*	Time
send-validity-yes	4	✓	✓	1.9s	×	2.1s
send-validity-no	4	×	×	1.9s	×	2.1s
receive-validity-yes	3	✓	✓	5.1s	×	2.3s
receive-validity-no	3	×	×	3.6s	×	2.0s
symbolic-two-bidder-yes	3	✓	✓	27.4s	×	2.0s
symbolic-two-bidder-no1	3	×	×	30.0s	×	2.1s
figure12-yes	3	✓	✓	2.0s	✓	2.0s
figure12-no	3	×	×	3.0s	✓	3.0s
symbolic-send-validity-yes	4	✓	✓	6.5s	×	2.5s
symbolic-send-validity-no	4	×	×	5.3s	×	2.6s
symbolic-receive-validity-yes	3	✓	✓	6.6s	×	2.8s
symbolic-receive-validity-no	3	×	×	7.6s	×	2.8s
fwd-auth-yes	3	✓	✓	10.3s	×	2.3s
fwd-auth-no	3	×	?	T/O	×	2.2s
symbolic-two-bidder-no2	3	×	×	23.9s	×	2.8s
higher-lower-ultimate	3	✓	✓	11.1s	×	2.4s
higher-lower-winning	3	✓	?	T/O	✓	229.8s
higher-lower-no	3	×	×	7.3s	N/A	2.2s
higher-lower-encrypt-yes	4	✓	✓	9.3s	×	2.3s
higher-lower-encrypt-no	4	×	×	177.3s	×	2.4s
higher-lower-mixed	3	×	×	19.3s	×	2.3s

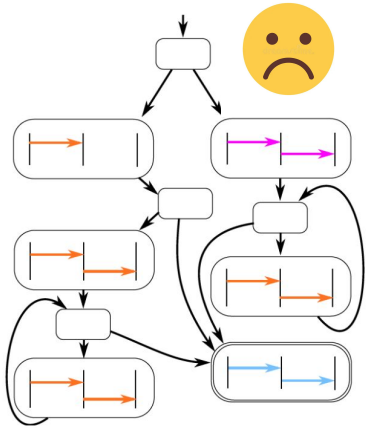
Explaining the completeness gap

- Syntactic approximation



- $\langle \mu t_1.(AL_1 \sqcap AL_2[t_2/t_1]), Msg_1 \cup Msg_2 \rangle$

$\text{if } L_1 = \mu t_1.AL_1,\ L_2 = \mu t_2.AL_2$



- $\langle \oplus_{i \in I}\, \mathsf{q}_i!m_i.(AL_{1,i} \sqcap AL_{2,i}), Msg_1 \cup Msg_2 \rangle$

$$\text{if } \begin{cases} L_1 = \oplus_{i \in I}\, \mathsf{q}_i!m_i.AL_{1,i}, \\ L_2 = \oplus_{i \in I}\, \mathsf{q}_i!m_i.AL_{2,i} \end{cases}$$

- Complexity gap: linear traversal vs. co-NP-complete decision problem

Explaining the completeness gap

- [Honda, Yoshida, Carbone 08] Multiparty asynchronous session types

Explaining the completeness gap

- [Honda, Yoshida, Carbone 08] Multiparty asynchronous session types
- [Bettini et al. 08] Global Progress in **Dynamically Interleaved** Multiparty Sessions

Explaining the completeness gap

- [Honda, Yoshida, Carbone 08] Multiparty asynchronous session types
- [Bettini et al. 08] Global Progress in Dynamically Interleaved Multiparty Sessions
- [Charalambides, Dinges, Agha 12] **Parameterized** concurrent multiparty session types

Explaining the completeness gap

- [Honda, Yoshida, Carbone 08] Multiparty asynchronous session types
- [Bettini et al. 08] Global Progress in Dynamically Interleaved Multiparty Sessions
- [Charalambides, Dinges, Agha 12] Parameterized concurrent multiparty session types
- [Scalas, Dardha, Hu, Yoshida 17] A linear decomposition of multiparty sessions for safe distributed programming
- [Toninho, Yoshida 17] Certifying **data** in multiparty session types
- [Zhou, Ferreira, Hu, Neykova, Yoshida 20] Statically Verified **Refinements** for Multiparty Protocols

Explaining the completeness gap

- [Honda, Yoshida, Carbone 08] Multiparty asynchronous session types
- [Bettini et al. 08] Global Progress in Dynamically Interleaved Multiparty Sessions
- [Charalambides, Dinges, Agha 12] Parameterized concurrent multiparty session types
- [Scalas, Dardha, Hu, Yoshida 17] A linear decomposition of multiparty sessions for safe distributed programming
- [Toninho, Yoshida 17] Certifying data in multiparty session types
- [Zhou, Ferreira, Hu, Neykova, Yoshida 20] Statically Verified Refinements for Multiparty Protocols
- [Le Brun, Dardha 23] MAGpi: Types for **Failure**-Prone Communication

Explaining the completeness gap

[Zhou et al. 20]

Example	$\lvert\mathcal{P}\rvert$	Impl.	SPROUT	Time	SESSION*	Time
send-validity-yes	4	✓	✓	1.9s	×	2.1s
send-validity-no	4	×	×	1.9s	×	2.1s
receive-validity-yes	3	✓	✓	5.1s	×	2.3s
receive-validity-no	3	×	×	3.6s	×	2.0s
symbolic-two-bidder-yes	3	✓	✓	27.4s	×	2.0s
symbolic-two-bidder-no1	3	×	×	30.0s	×	2.1s
figure12-yes	3	✓	✓	2.0s	✓	2.0s
figure12-no	3	×	×	3.0s	✓	3.0s
symbolic-send-validity-yes	4	✓	✓	6.5s	×	2.5s
symbolic-send-validity-no	4	×	×	5.3s	×	2.6s
symbolic-receive-validity-yes	3	✓	✓	6.6s	×	2.8s
symbolic-receive-validity-no	3	×	×	7.6s	×	2.8s
fwd-auth-yes	3	✓	✓	10.3s	×	2.3s
fwd-auth-no	3	×	?	T/O	×	2.2s
symbolic-two-bidder-no2	3	×	×	23.9s	×	2.8s
higher-lower-ultimate	3	✓	✓	11.1s	×	2.4s
higher-lower-winning	3	✓	?	T/O	✓	229.8s
higher-lower-no	3	×	×	7.3s	N/A	2.2s
higher-lower-encrypt-yes	4	✓	✓	9.3s	×	2.3s
higher-lower-encrypt-no	4	×	×	177.3s	×	2.4s
higher-lower-mixed	3	×	×	19.3s	×	2.3s

…and the soundness gap

A system of communicating finite state machines is *synchronizable* [1, 4] if its send trace semantics, i.e. the set of sequences of sendings it can perform, is the same when its communications are FIFO asynchronous and when they are just rendez-vous synchronizations. This property was claimed to be decidable in several conference and journal papers [1, 4, 3, 2] for either mailboxes (∗-1) or peer-to-peer (1-1) communications, thanks to a form of small model property. In this paper, we show that this small model property does not hold neither for mailbox communications, nor for peer-to-peer communications, therefore the decidability of synchronizability becomes an open question. We close this question for peer-to-peer communications, and we show that synchronizability is actually undecidable. We show that synchronizability is decidable if the topology

[Finkel and Lozes 17]

(b)	Caires and Pérez [2016]; Chen [2015]; Deniélou et al. [2012]; Deniélou and Yoshida [2012]; Toninho and Yoshida [2016]	≥ full	no	flawed	(C2)

[Scalas and Yoshida 19]

Contributions

- Implementability:
  - Network-parametric, precise characterization for infinite-state protocols (GCLTS)
  - Optimal decision procedures for finite fragments
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - **Rocq mechanization**
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Rocq mechanization [Li and Wies ITP'25]

```
Theorem preciseness :
  ∀ {State : Type}
    {ChanOp : @ChannelArch Channel message M}
    {CC : @ChannelComplianceFacts message M}
    {R : @CLTSCCBridge Channel message M ChanOp CC}
    (S : @LTS SyncAlphabet State),
    GCLTS S →
    (∃ (LocalState : Type),
        @implementable State Channel LocalState message M ChanOp CC S) ↔
      @NMC State message S ∧ @RCC State message M CC S ∧ @SCC State message S.
```

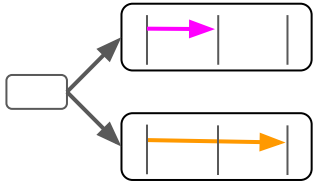
operational
algebraic
correspondence

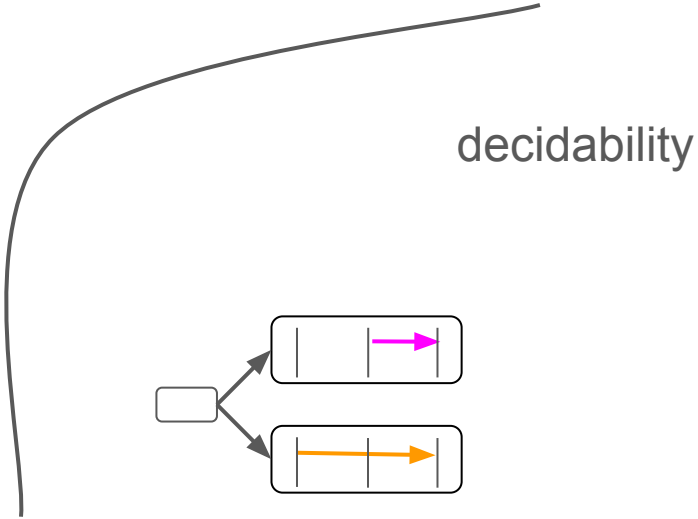
Takeaways:

- Generalization to infinite participants
- Sink-finality GCLTS assumption can be conditionally removed
- Infinite word semantics bug

GCLTS assumptions

1) Deadlock-free
2) Deterministic
3) Sender-driven choice





4) Sink-finality: no final states with outgoing transitions

cf. "local acceptance" for Zielonka automata, only required for finite CLTS semantics

Rocq mechanization [Li and Wies ITP'25]

```
Theorem preciseness :
  ∀ {State : Type}
    {ChanOp : @ChannelArch Channel message M}
    {CC : @ChannelComplianceFacts message M}
    {R : @CLTSCCBridge Channel message M ChanOp CC}
    (S : @LTS SyncAlphabet State),
    GCLTS S →
    (∃ (LocalState : Type),
        @implementable State Channel LocalState message M ChanOp CC S) ↔
      @NMC State message S ∧ @RCC State message M CC S ∧ @SCC State message S.
```

operational
algebraic
correspondence

Takeaways:

- Generalization to infinite participants
- Sink-finality GCLTS assumption can be conditionally removed
- **Infinite word semantics bug**

Global semantics must be indistinguishability-closed

Finite words:

$$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot s ? b_1 : y \cdot b_2 ? b_1 : y \cdot s ! b_1 : z \quad \checkmark$$

$$b_1 ! s : y \cdot b_1 ! b_2 : y \cdot b_2 ? b_1 : y \cdot s ? b_1 : y \cdot s ! b_1 : z \quad \checkmark$$

Global semantics must be indistinguishability-closed

Infinite words:

$$\mathsf{p}\,!\,\mathsf{q}:m^n \cdot \mathsf{p}\,!\,\mathsf{q}:m' \cdot \mathsf{r}\,!\,\mathsf{s}:m' \qquad \text{for all } n \in \mathbb{N} \quad \checkmark$$

$$\mathsf{r}\,!\,\mathsf{s}:m' \cdot \mathsf{p}\,!\,\mathsf{q}:m^\omega \quad \checkmark$$

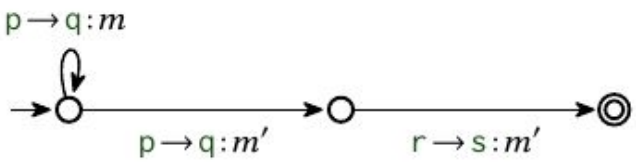
Global semantics must be indistinguishability-closed

Infinite words:

$$\mathsf{p}\,!\,\mathsf{q}:m^n \cdot \mathsf{p}\,!\,\mathsf{q}:m' \cdot \mathsf{r}\,!\,\mathsf{s}:m' \qquad \text{for all } n \in \mathbb{N}$$

$$\mathsf{r}\,!\,\mathsf{s}:m' \cdot \mathsf{p}\,!\,\mathsf{q}:m^\omega$$

"there exists a global run for every prefix of an infinite word" [Majumdar et al. 22]



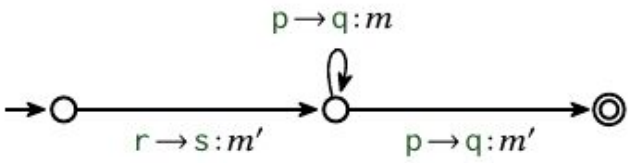
Global semantics must be indistinguishability-closed

Infinite words:

$$\mathsf{p}\,!\,\mathsf{q}:m^n \cdot \mathsf{p}\,!\,\mathsf{q}:m' \cdot \mathsf{r}\,!\,\mathsf{s}:m' \qquad \text{for all } n \in \mathbb{N}$$

$$\mathsf{r}\,!\,\mathsf{s}:m' \cdot \mathsf{p}\,!\,\mathsf{q}:m^\omega$$

"there exists a global run for every prefix of an infinite word" [Majumdar et al. 22]



Global semantics must be indistinguishability-closed

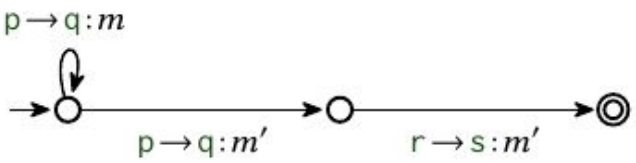
Infinite words:

$$\mathsf{p}\,!\,\mathsf{q}:m^n \cdot \mathsf{p}\,!\,\mathsf{q}:m' \cdot \mathsf{r}\,!\,\mathsf{s}:m' \qquad \text{for all } n \in \mathbb{N}$$

$$\mathsf{r}\,!\,\mathsf{s}:m' \cdot \mathsf{p}\,!\,\mathsf{q}:m^\omega$$

"there exists a global run for every prefix of an infinite word" [Majumdar et al. 22]

"for every prefix of an infinite word there exists a global run" [Li and Wies ITP'25]



Also in the thesis

- Implementability:
  - Network-parametric, precise characterization for infinite-state protocols (GCLTS)
  - Optimal decision procedures for finite fragments
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - Rocq mechanization
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Acknowledgements

Contributions

- Implementability:
  - Precise characterization for infinite-state protocols (GCLTS)
  - Optimal decision procedures for finite fragments
  - Sound and relatively complete algorithm for symbolic protocols, implemented in SPROUT
  - Network architecture-parametric characterization for GCLTS
- Synthesis:
  - Sound and complete algorithm for finite protocols
  - Blueprint for infinite protocols
- Subtyping:
  - Sound and complete algorithms for finite protocols

Thank you!